



A small dive into Dhrystone & CoreMark

SHAKTI Group | CSE Dept | PS-CDISHA - RISE Lab | IIT Madras

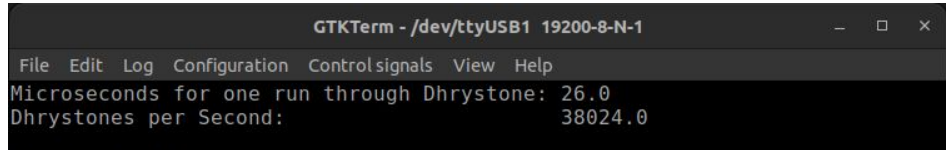
What is Dhrystone?

- Dhrystone is a free, synthetic benchmark used to measure general-purpose processor performance, particularly in embedded systems.
- Created in 1984 by Reinhold Weicker, it became an industry-standard benchmark for evaluating integer and string operation performance before being succeeded by CoreMark.

Let us Dhrystone

Command to run

```
$ make software PROGRAM=dhrystone TARGET=yamuna
```



A screenshot of a terminal window titled "GTKTerm - /dev/ttyUSB1 19200-8-N-1". The window has a menu bar with "File", "Edit", "Log", "Configuration", "Controlsignals", "View", and "Help". The terminal output shows the results of the Dhrystone benchmark:

```
Microseconds for one run through Dhrystone: 26.0  
Dhrystones per Second: 38024.0
```

How does Dhrystone work?

- **Workload Includes:**

- String handling and manipulation (character and memory operations).
- Integer arithmetic and logic operations.
- Control flow (loops, conditionals, and procedure calls).
- Struct and pointer operations simulating system-level workloads.
- Portable: Written in C, ensuring compatibility across platforms.

- **Key Metrics:**

- **Dhrystones/Second:** Number of times the benchmark loop executes per second.
- **DMIPS (Dhrystone MIPS):** Performance metric normalized against a VAX 11/780 computer (1 DMIPS = 1757 Dhrystones/second).

Scores of popular CPUs

CPU	Clock (MHz)	Dhrystone 2 Opt (VAX)	DMIPS/MHz
AMD 80386	40	13.7	0.34 ^[1]
Pentium 100	100	122	1.22 ^[1]
Pentium III 1000	1000	1595	1.60 ^[1]
Pentium 4 3066	3066	4012	1.31 ^[1]
Core i5 2467M	2467	4752	1.93 ^[1]

Sources: [1]R. Longbottom, "Dhrystone Benchmark Results on PCs and Later Devices," Online – ResearchGate, 2017. Available:
https://www.researchgate.net/publication/319176416_Dhrystone_Benchmark_Results_On_PC's_and_Later_Devices
Roy Longbottom

Dhrystone

Weicker, R. P. (1984). Dhrystone: A synthetic systems programming benchmark. *Communications of the ACM*, 27(10), 1013–1030. <https://doi.org/10.1145/358274.358283>

Should we use Dhrystone?

Pros

- **Standardization:** Provides a consistent metric for comparing integer performance across processors.
- **Simplicity:** Small, portable C code (~2,000 lines for Dhrystone 2) that is easy to compile and run.
- **Transparency:** Widely published results and open access make it easy to verify and reproduce.
- **Historical Benchmark:** Long history allows comparison across decades of CPUs.

Cons

- **Narrow Focus:** Measures mostly integer arithmetic and string/logic operations; ignores floating-point, memory/cache performance, I/O, and multicore scaling.
- **Synthetic Workload:** Does not necessarily reflect real-world application performance; results can be heavily affected by compiler optimizations.
- **Obsolete for Modern CPUs:** Modern superscalar, out-of-order, and multi-core architectures are not fully stressed by Dhrystone.
- **Compiler Sensitivity:** Scores can vary significantly depending on compiler version and optimization flags.

What is CoreMark?

- CoreMark is a free, industry-standard benchmark for measuring embedded processor performance.
- Created by the Embedded Microprocessor Benchmark Consortium (EEMBC) to replace Dhrystone.

Let us CoreMark

Command to run

```
$ make software PROGRAM=coremarks TARGET=yamuna
```

```
GTKTerm - /dev/ttyUSB1 19200-8-N-1
File Edit Log Configuration Controlsignals View Help
Microseconds for one run through Dhrystone: 26.0
Dhrystones per Second: 38024.0

Start Time print
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 35444949
Total time (secs): 35
Iterations/Sec : 2
Iterations : 100
Compiler version : riscv32-unknown-elf-13.2.0
Compiler flags : -O3 -fno-common -funroll-loops -finline-functions -fsel
ective-scheduling -falign-functions=16 -falign-jumps=4 -falign-loops=4 -fi
nline-limit=1000 -nostartfiles -nostdlib -ffast-math -fno-builtin-printf -
march=rv32imac zicsr -mexplicit-relocs
Memory location : STACK
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0x988c
Correct operation validated. See README.md for run and reporting rules.

Exception: mcause = 1, epc = 0
█

/dev/ttyUSB1 19200-8-N-1 DTR RTS CTS CD DSR RI
```


How does CoreMark work?

- **Workload Includes:**
 - Linked list traversal (memory access).
 - Matrix multiplication (integer operations).
 - State machine (branching logic).
 - CRC (cyclic redundancy check for data integrity).
- **Portable:** Written in C, ensuring compatibility across platforms.
- **Key Metrics:**
 - **CoreMarks=Iterations/Second:** Number of times the benchmark runs per second.
 - **CoreMark/MHz:** Normalized score accounting for clock speed, enabling cross-architecture comparisons.

Scores of popular CPUs

CPU	Core Count	Clock Frequency (GHz)	Score (CM/MHz)
i7 10510U	4	4.9	22.96 ^[1]
i5 12500H	16	4.5	71.95 ^[1]
Ryzen 9 7900X	12	4.7	176.8 ^[1]
SiFive HiFive U	4	1.5	2.01 ^[1]
ARM Cortex A53	4	1.5	13.12 ^[1]
SonicBOOM	1	3.2	6.2 ^[2]
Rocket	1	1.5	2.32 ^[2]

Sources: [1] EEMBC, [2] Krste Asanovic, David A Patterson, and Christopher Celio. 2015. The Berkeley out of-order machine (boom): An industry-competitive, synthesizable, parameterized RISC-V processor. Technical Report. University of California at Berkeley, Berkeley, United States.

Should we use CoreMark?

1. Pros:

- a. **Standardization:** Enables apples-to-apples comparisons between processors.
- b. **Simplicity:** Small code footprint (~1,200 lines) and easy to run.
- c. **Transparency:** Open-source and free to use.

2. Cons:

- a. **Narrow Focus:** Only focuses on pipeline efficiency and does not measure memory/cache performance, I/O, power consumption, or multicore scaling.
- b. **Synthetic:** May not reflect real-world application performance.

Thank you

Website: shakti.org.in

GitLab: gitlab.com/shaktiproject/cores/c-class